

# Week 4, Part 2: Flutter Architecture and Widget Types

Dr M AOUE  
Lecture notes

**Objectives.** This is the second part of week 4. By the end you will be able to describe the layers of the Flutter architecture; explain what the widget tree is, and print the real tree of an app; tell visible widgets from layout widgets and use the most common ones; react to touches with `GestureDetector`; and explain when a widget must be stateful.

Three examples of the project `flutter_examples` (in `code/week04`) go with this part:

- `13_widget_types`: visible and layout widgets;
- `14_gestures`: touches and state;
- `15_widget_tree_dump`: the widget tree printed by Flutter.

The listings below are taken from those files, and automated tests check what they show. See the guide *Running the Flutter apps in VS Code* to run them.

## 1 The architecture of Flutter

Flutter is a layered system. Each layer only relies on the layer below it, and most layers can be replaced. There are three main layers:

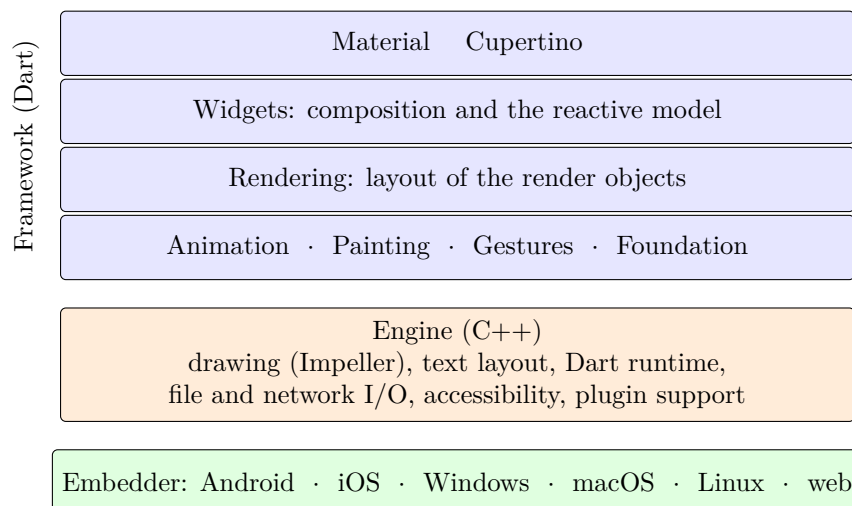


Figure 1: The layers of Flutter, from the widgets you write (top) to the operating system (bottom).

| Layer            | Written in                    | Role                                                                                                              |
|------------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Framework</b> | Dart                          | everything you use to write an app: the widgets, layout, animation, gestures                                      |
| <b>Engine</b>    | mostly C++                    | draws the scenes on the screen, runs Dart code, provides text layout, file and network I/O, accessibility         |
| <b>Embedder</b>  | the language of each platform | starts the app on a given operating system and gives it a surface to draw on, input events and the app life cycle |

### The four components of the lecture

The lecture names four components of the architecture. Here is where each one is in the picture:

| Component                      | In the layers                                                                                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Flutter Engine</b>          | the <i>Engine</i> layer: a portable runtime, mostly C++, with graphics, text, file and network I/O, a plugin architecture, accessibility support and the Dart runtime |
| <b>Foundation library</b>      | the bottom of the <i>Framework</i> : basic classes written in Dart (foundation, animation, painting, gestures) on which everything else is built                      |
| <b>Widgets</b>                 | the <i>Widgets</i> layer (and the <i>Rendering</i> layer below it): the composition of the user interface                                                             |
| <b>Design-specific widgets</b> | the <i>Material</i> and <i>Cupertino</i> libraries: complete sets of controls in two design languages                                                                 |

### Two things that changed.

- **The drawing library.** Older texts say that Flutter draws with Skia. Today the default renderer is **Impeller**: on iOS, on Android (API 29 and later, since Flutter 3.27), and on Windows, macOS and Linux (since Flutter 3.47). The web still uses Skia.
- **Material and Cupertino.** They are *not* tied to Android and iOS. Flutter draws every control itself, so both libraries work on every platform, and an app looks the same on all versions of the operating system. You choose the style you want; you can also choose it per platform.

## 2 Everything is a widget: the widget tree

In Flutter you *describe* the user interface with widgets, and Flutter draws it. A widget is an **immutable description** of a part of the screen: text, a shape, an image, a layout, an animation, a style. Widgets are nested inside each other: the root of the app is a widget, and everything below it is a widget too. This nesting is the **widget tree**.

- The **build** method of a widget returns the widgets that describe it. It must be fast, and free of side effects, because Flutter may call it at any time.
- When the state of the app changes, Flutter builds the new tree, **compares** it with the old one, and changes only the parts of the screen that differ. You never edit the screen yourself.
- For layout, the rule is: *constraints go down, sizes go up*. A parent tells each child how much room it may use; the child chooses its size within that room and tells the parent.

### The tree of a real app

The app of `15_widget_tree_dump` has a `Scaffold` with an `AppBar`, two lines of text and a button. This is its tree, as printed by Flutter (a few widgets are left out):

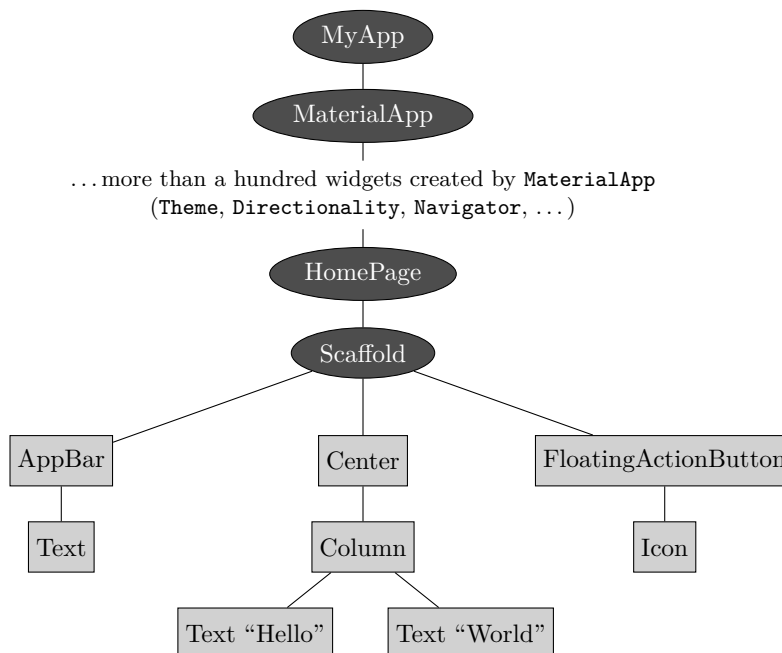


Figure 2: The widget tree of the example. Dark ovals: widgets that build other widgets. Grey boxes: the widgets inside the `Scaffold`.

Only about a dozen of these widgets appear in our code; the full dump of this tiny app lists more than two hundred. `MaterialApp` and `Scaffold` create all the others (themes, text direction, navigation, ink effects, ...). The button below calls `debugDumpApp()`, which prints the tree in the console of the terminal or of VS Code's Debug Console:

```

1 // The root of the tree.
2 class MyApp extends StatelessWidget {
3   const MyApp({super.key});
4
5   @override
6   Widget build(BuildContext context) {
7     return const MaterialApp(home: HomePage());
8   }
9 }
10
11 // The page: a Scaffold with an app bar, a body and a button.
12 class HomePage extends StatelessWidget {
13   const HomePage({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     return Scaffold(
18       appBar: AppBar(title: const Text('Widget tree')),
19       body: const Center(
20         child: Column(
21           mainAxisAlignment: MainAxisAlignment.min,
22           children: [Text('Hello'), Text('World')],
23         ),
24       ),
25       floatingActionButton: FloatingActionButton(
26         // Print the whole widget tree of the running app in the console.

```

```

27     onPressed: () {
28       debugPrint('--- widget tree of the running app ---');
29       debugDumpApp();
30     },
31     child: const Icon(Icons.account_tree),
32   ),
33 );
34 }
35 }

```

In the dump, each widget appears after the widget that contains it, indented one level deeper. A simplified extract:

```

MyApp
  MaterialApp
    ... (Directionality, Theme, Navigator, ...)
    HomePage
      Scaffold
        ...
        Center          <- the body
          Column
            Text("Hello")
            Text("World")
          AppBar         <- the appBar
            ... Text("Widget tree")
        FloatingActionButton
          ... Icon

```

### 3 Widget types

Widgets can be divided into two families:

- **Visible widgets** show something or take input from the user: `Text`, buttons, `Image`, `Icon`.
- **Layout (invisible) widgets** draw nothing themselves; they place and size other widgets: `Column`, `Row`, `Center`, `Padding`, `Stack`, `Scaffold`.

#### Visible widgets

- **Text** shows text. `textAlign` aligns it, and `style` takes a `TextStyle` (font, size, weight, color, spacing, ...).
- **Buttons** run a function when tapped. The three main ones are `ElevatedButton`, `TextButton` and `OutlinedButton`. Each has a `child` (its label) and an `onPressed` function. Old tutorials use `RaisedButton`, `FlatButton` and `OutlineButton`: they were replaced in Flutter 2 by `ElevatedButton`, `TextButton` and `OutlinedButton`, and should not be used any more.
- **Image** shows a picture. There is one constructor per source:

| Constructor                | Loads the image from                      |
|----------------------------|-------------------------------------------|
| <code>Image.asset</code>   | a file of your project (an <i>asset</i> ) |
| <code>Image.network</code> | a URL on the Internet                     |
| <code>Image.file</code>    | a file on the device                      |
| <code>Image.memory</code>  | bytes already in memory                   |

To use an asset, put the file in a folder of the project (for example `assets`) and declare the folder in `pubspec.yaml`, with the same indentation as below:

```
flutter:
  assets:
    - assets/
```

then write `Image.asset('assets/logo.png')`.

- **Icon** shows one of the built-in icons (`Icons.favorite, ...`) with a **size** and a **color**.

### Layout widgets

- **Column** and **Row** place their children vertically or horizontally. The *main axis* is the direction of the widget (vertical for a **Column**); the *cross axis* is the other one. **mainAxisAlignment** and **crossAxisAlignment** say how the children are spread along each axis.
- **Center** centers its child. **Padding** adds space around its child (`EdgeInsets.all(8)` adds 8 logical pixels on every side).
- **Stack** draws its children on top of each other, the first one at the bottom. **Scaffold** gives a page its structure: app bar, body, buttons, drawers.

The gallery below shows all of them in one page. You do not write **new** before a widget: Dart does not need it.

```
1 // Stateful, because the page remembers which button was tapped last.
2 class WidgetGallery extends StatefulWidget {
3   const WidgetGallery({super.key});
4
5   @override
6   State<WidgetGallery> createState() => _WidgetGalleryState();
7 }
8
9 class _WidgetGalleryState extends State<WidgetGallery> {
10   String _lastButton = 'none';
11
12   void _tapped(String name) => setState(() => _lastButton = name);
13
14   @override
15   Widget build(BuildContext context) {
16     return Padding(
17       padding: const EdgeInsets.all(16), // space around the whole page
18       child: Column(
19         children: [
20           // ----- Visible widgets -----
21           const Text(
22             'Hello, Flutter!',
23             textAlign: TextAlign.center,
24             style: TextStyle(fontWeight: FontWeight.bold, fontSize: 20),
25           ),
26           Row(
27             mainAxisAlignment: MainAxisAlignment.spaceEvenly,
28             children: [
29               ElevatedButton(
30                 onPressed: () => _tapped('ElevatedButton'),
31                 child: const Text('Elevated'),
32               ),
33               TextButton(
34                 onPressed: () => _tapped('TextButton'),
35                 child: const Text('Text'),
36               ),
37               OutlinedButton(
38                 onPressed: () => _tapped('OutlinedButton'),
39                 child: const Text('Outlined'),
40               ),
41             ],
42           ),
43         ],
44       ),
45     );
46   }
47 }
```

```

41     ],
42   ),
43   // An image from the project's assets folder.
44   Image.asset('assets/logo.png', width: 64, height: 64),
45   const Icon(Icons.favorite, color: Colors.red, size: 35),
46   Text('Last button: $_lastButton'),
47
48   // ----- Layout widgets -----
49   const SizedBox(height: 24),
50   // Row: side by side. Padding: space around each label.
51   const Row(
52     mainAxisAlignment: MainAxisAlignment.spaceEvenly,
53     children: [
54       Padding(padding: EdgeInsets.all(8), child: Text('Row 1')),
55       Padding(padding: EdgeInsets.all(8), child: Text('Row 2')),
56       Padding(padding: EdgeInsets.all(8), child: Text('Row 3')),
57     ],
58   ),
59   // Stack: the children are drawn on top of each other, in order.
60   Stack(
61     alignment: Alignment.center, // centre the smaller ones
62     children: [
63       Container(width: 120, height: 120, color: Colors.blue),
64       Container(width: 70, height: 70, color: Colors.red.shade300),
65       const Text('Stack', style: TextStyle(color: Colors.white)),
66     ],
67   ),
68 ],
69 ),
70 );
71 }
72 }

```

## 4 Gestures

`GestureDetector` is an *invisible* widget that makes its child interactive. It listens for taps, long presses, drags and scaling (pinching), and calls the function you give it: `onTap`, `onLongPress`, `onPanUpdate`, ... The child decides what is seen and how big the touch area is. In the example a tap adds 1 to a counter and a long press resets it.

```

1 class TapCounterBox extends StatefulWidget {
2   const TapCounterBox({super.key});
3
4   @override
5   State<TapCounterBox> createState() => _TapCounterBoxState();
6 }
7
8 class _TapCounterBoxState extends State<TapCounterBox> {
9   int _taps = 0; // the state: it changes while the app runs
10
11   @override
12   Widget build(BuildContext context) {
13     return Column(
14       mainAxisAlignment: MainAxisAlignment.min,
15       children: [
16         // The GestureDetector listens to touches on everything inside it.
17         GestureDetector(
18           key: const ValueKey('touch-area'),
19           onTap: () => setState(() => _taps++),
20           onLongPress: () => setState(() => _taps = 0),
21           child: Stack(

```

```

22         children: [
23             Container(
24                 width: 200,
25                 height: 200,
26                 color: Colors.indigo.shade100,
27                 alignment: Alignment.center,
28                 child:
29                     Text('Taps: $_taps', style: const TextStyle(fontSize: 24)),
30             ),
31             // Positioned puts the icon in the top-right corner of the Stack.
32             const Positioned(
33                 top: 8,
34                 right: 8,
35                 child: Icon(Icons.touch_app),
36             ),
37         ],
38     ),
39 ],
40 const SizedBox(height: 12),
41 const Text('Tap = +1, long press = reset'),
42 ],
43 );
44 }
45 }

```

## 5 State management: stateless and stateful widgets

A widget is **stateless** if nothing in it changes while the app runs: it only turns its inputs into a piece of the screen. A widget is **stateful** if it keeps data that changes, and must be drawn again when the data changes.

- A **StatelessWidget** has one class, with a **build** method. **Text**, **Icon** and **Container** are stateless.
- A **StatefulWidget** is made of *two* classes. The widget itself is immutable and only has a method **createState()**. It returns a class that extends **State**, which keeps the changing data and has the **build** method.
- To change the data you call **setState(() { ... })**. Flutter runs the function and builds the widget again; then, as usual, only the parts that changed are redrawn.
- **Checkbox**, **Radio**, **Slider**, **Form** and **TextField** are stateful.

**Row**, **Column**, **Center** and **Padding** are layout widgets: they are not built from a **build** method like the stateless widgets you write, but they have no state of their own either. When you write your own widget, ask: *does it have data that changes while the app runs?* If yes, make it stateful. The examples **00\_stateless\_vs\_stateful** (week 4) and **14\_gestures** show both kinds.

Data that must be shared by many widgets, such as the theme or the size of the screen, is passed down the tree by a special widget called an **InheritedWidget**: this is how **Theme.of(context)** and **MediaQuery.of(context)** work. We will use this idea again in later weeks.

## 6 Summary

| Idea                                | In one sentence                                                                                                                  |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Layers</b>                       | Embedder (per platform), Engine (C++), Framework (Dart)                                                                          |
| <b>Widget</b>                       | an immutable description of a part of the user interface                                                                         |
| <b>Widget tree</b>                  | the widgets nested in each other, from the root <code>MyApp</code> down to the leaves; <code>debugDumpApp()</code> prints it     |
| <b>Visible widgets</b>              | <code>Text</code> , buttons, <code>Image</code> , <code>Icon</code>                                                              |
| <b>Layout widgets</b>               | <code>Column</code> , <code>Row</code> , <code>Center</code> , <code>Padding</code> , <code>Stack</code> , <code>Scaffold</code> |
| <b><code>GestureDetector</code></b> | an invisible widget that listens to touches on its child                                                                         |
| <b>Stateless / stateful</b>         | fixed content / content that changes; <code>setState</code> rebuilds a stateful widget                                           |

---

This document is maintained by Dr. Mohamad AOUE

Last updated: 2026-09-21